310. Minimum Height Trees Medium

Acceptance: 42.4% | LeetCode

A tree is an undirected graph in which any two vertices are connected by *exactly* one path. In other words, any connected graph without simple cycles is a tree.

Given a tree of `n` nodes labelled from `0` to `n - 1`, and an array of `n - 1` `edges` where `edges[i] = [ai, bi]` indicates that there is an undirected edge between the two nodes `ai` and `bi` in the tree, you can choose any node of the tree as the root. When you select a node `x` as the root, the result tree has height `h`. Among all possible rooted trees, those with minimum height (i.e. `min(h)`) are called **minimum height trees** (MHTs).

Return *a list of all**MHTs**' root labels*. You can return the answer in **any order**.

The **height** of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:

```
**Input:** n = 4, edges = [[1,0],[1,2],[1,3]]
**Output:** [1]
**Explanation:** As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.
```

Example 2:

```
**Input:** n = 6, edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]
**Output:** [3,4]
```

Constraints:

- `1 <= n <= 2 * 104`
- `edges.length == n - 1`
- `0 <= ai, bi < n`
- `ai != bi`
- All the pairs `(ai, bi)` are distinct.
- The given input is **guaranteed** to be a tree and there will be **no repeated** edges.

Claude Opus Solution

Minimum Height Trees

1. Intuition

The key insight: The optimal roots (MHT roots) are the **center(s)** of the tree.

Think about it geometrically — the longest path in a tree is called its **diameter**. If you root the tree at an endpoint of this diameter, you get the maximum possible height. To minimize the height, you want to root at the **middle** of the longest path, so the tree spreads out as evenly as possible.

Critical fact: A tree can have **at most 2 centers**. - If the diameter (number of nodes on the longest path) is **odd**, there's exactly **1 center**. - If the diameter is **even**, there are exactly **2 centers** (the two middle nodes).

This is why the answer always has 1 or 2 nodes.

The "onion peeling" trick: Instead of computing the diameter directly, we can repeatedly remove leaf nodes layer by layer (like peeling an onion). The last 1 or 2 nodes remaining are the centers — they're the deepest "inside" nodes, equidistant from all extremes.

---

2. Approach (Pseudocode)

```
1. Handle edge cases: if n == 1 (no edges), return [0]

2. Build an adjacency list (using sets for O(1) removal)

3. Identify all initial leaves (nodes with degree == 1)

4. Topological "trimming" loop:
   While more than 2 nodes remain:
       a. For each current leaf:
          - Remove it from the graph
          - Find its single neighbor; remove edge to leaf
          - If neighbor now has degree 1, it becomes a new leaf
       b. Replace current leaves with the new leaves

5. Return the remaining 1 or 2 nodes (the centers)
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        # ---- Edge Case ----
        # A single node tree (n == 1) has no edges; node 0 is the only root.
        # Height is 0, so it's trivially a minimum height tree.
        if n == 1:
            return [0]

        # ---- Step 1: Build adjacency list ----
        # We use sets because we need O(1) removal of neighbors later.
        adjacency = [set() for _ in range(n)]
        for a, b in edges:
            adjacency[a].add(b)
            adjacency[b].add(a)

        # ---- Step 2: Find initial leaves ----
        # A leaf is a node with exactly one neighbor (degree == 1).
        leaves = [node for node in range(n) if len(adjacency[node]) == 1]

        # ---- Step 3: Peel layers until <= 2 nodes remain ----
        # `remaining` tracks how many nodes are still in the tree.
        remaining = n
        while remaining > 2:
            # We will remove all current leaves this round.
            remaining -= len(leaves)
            new_leaves = []

            # Process each leaf in the current layer.
            for leaf in leaves:
                # Each leaf has exactly ONE neighbor; pop it out.
                neighbor = adjacency[leaf].pop()
                # Remove the back-edge so neighbor's degree decreases.
                adjacency[neighbor].remove(leaf)
                # If the neighbor just became a leaf (degree 1),
                # it will be peeled in the next round.
                if len(adjacency[neighbor]) == 1:
                    new_leaves.append(neighbor)

            # Move to the next layer of leaves.
            leaves = new_leaves

        # ---- Step 4: The remaining 1 or 2 nodes are the centers ----
        return leaves
```

---

4. Complexity Analysis

Time Complexity: O(N) - Building the adjacency list takes O(N) since a tree has exactly `n - 1` edges. - In the trimming loop, each node is added to `leaves` exactly once and processed exactly once. Each edge is removed exactly once. So the total work across all loop iterations is O(N).

Space Complexity: O(N) - The adjacency list stores `2 * (n - 1)` entries total → O(N). - The `leaves` and `new_leaves` lists hold at most O(N) nodes.

---

5. Alternative Approaches

A) Brute Force — BFS/DFS from every node ❌

For each of the `n` nodes, run a BFS to compute the tree's height when rooted there, then pick the minimum. - **Time:** $O(N^2)$ — too slow for $n = 2 \times 10^4$. - Simple but inefficient; gets TLE on large inputs.

B) Two BFS to Find Diameter Endpoints, then Walk to Center ✅

1. BFS from any node to find the farthest node `u` (one endpoint of the diameter).
2. BFS from `u` to find the farthest node `v` (other endpoint), recording the path.

3. The center(s) of the path from `u` to `v` are the MHT roots.
4. **Time:** O(N), **Space:** O(N).
5. Mathematically elegant but trickier to implement (need to reconstruct the path).

C) Topological Trimming (Our Solution) ✅ — Recommended

- Cleanest to implement, O(N) time, naturally returns 1 or 2 centers.
- Conceptually a **multi-source BFS** peeling from all leaves simultaneously — equivalent to a topological sort using degree (Kahn's algorithm idea, adapted for undirected trees).

Key Pattern Takeaway

> **"Centers via leaf-trimming"** is a powerful technique whenever you need a balanced/central node in a tree. When you see "*minimize maximum distance*" in a tree, think **diameter** → **center**, and the BFS-from-leaves trimming is the simplest way to find it.

Codex Solution

1. Intuition

A **Minimum Height Tree root** must be as close as possible to all leaves.

In a tree, the longest paths always end at leaves. So if we repeatedly remove all current leaves, we are trimming the tree from the outside inward.

The last remaining node or two nodes are the **center(s)** of the tree.

Those centers are exactly the roots that produce minimum height trees.

Key facts:

- A tree can have either **1 center** or **2 centers**.
- Removing leaves layer by layer does not change the eventual center.
- This is similar to **topological sorting**, but on an undirected tree.

Example:

```
0 - 1 - 2 - 3 - 4
```

Remove leaves:

```
0, 4 removed
1 - 2 - 3
```

Remove leaves again:

```
1, 3 removed
2 remains
```

So node `2` is the only MHT root.

For an even-length center:

```
0 - 1 - 2 - 3
```

Remove leaves:

```
0, 3 removed
1 - 2 remain
```

So both `1` and `2` are MHT roots.

---

2. Approach

We use **BFS-style leaf trimming**.

Pseudocode

```
if n == 1:
    return [0]
```

```
build adjacency list
build degree array

initialize queue with all leaf nodes
    leaf node = node with degree 1

remaining_nodes = n

while remaining_nodes > 2:
    current_leaf_count = size of queue
    remaining_nodes -= current_leaf_count

    repeat current_leaf_count times:
        leaf = pop from queue

        for neighbor in adjacency[leaf]:
            degree[neighbor] -= 1

            if degree[neighbor] == 1:
                add neighbor to queue

return nodes remaining in queue
```

---

3. Python Solution

```python
from typing import List
from collections import deque


class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        # Edge case:
        # If there is only one node, it is trivially the root of the MHT.
        if n == 1:
            return [0]

        # Build adjacency list.
        # adjacency[node] contains all neighbors of node.
        adjacency = [[] for _ in range(n)]

        # degree[node] stores how many edges are connected to node.
        # Leaves are nodes with degree 1.
        degree = [0] * n

        for a, b in edges:
            adjacency[a].append(b)
            adjacency[b].append(a)

            degree[a] += 1
            degree[b] += 1

        # Initialize queue with all current leaves.
        leaves = deque()

        for node in range(n):
            if degree[node] == 1:
                leaves.append(node)

        # Number of nodes that are still not removed.
        remaining_nodes = n

        # Repeatedly remove leaves until only 1 or 2 nodes remain.
        #
        # Why stop at 2?
        # A tree can have at most two centers.
        # Those final center nodes are the MHT roots.
        while remaining_nodes > 2:
            # Remove one full "layer" of leaves.
            current_leaf_count = len(leaves)
            remaining_nodes -= current_leaf_count
```

```
            for _ in range(current_leaf_count):
                leaf = leaves.popleft()

                # "Remove" this leaf by reducing the degree of its neighbors.
                for neighbor in adjacency[leaf]:
                    degree[neighbor] -= 1

                    # If a neighbor becomes degree 1, it is a new leaf
                    # for the next trimming round.
                    if degree[neighbor] == 1:
                        leaves.append(neighbor)

        # The remaining node(s) are the tree center(s),
        # therefore they are the roots of Minimum Height Trees.
        return list(leaves)
```

---

4. Complexity Analysis

Let `n` be the number of nodes.

Since the input is a tree, there are exactly `n - 1` edges.

Time Complexity: `O(n)`

Building the graph takes:

```
O(n)
```

because there are `n - 1` edges.

During leaf trimming:

- Each node is pushed into the queue at most once.
- Each edge is processed at most twice, once from each endpoint.

So the total work is:

```
O(n)
```

Therefore:

```
Total Time Complexity = O(n)
```

Space Complexity: `O(n)`

We store:

- adjacency list: `O(n)`
- degree array: `O(n)`
- queue of leaves: `O(n)`

Therefore:

```
Total Space Complexity = O(n)
```

---

5. Alternative Approaches

1. Tree Diameter Method

Another common solution uses the fact that MHT roots are the middle node or nodes of the tree's **diameter**.

Steps:

1. Pick any node and run BFS/DFS to find the farthest node `A`.
2. Run BFS/DFS from `A` to find the farthest node `B`.
3. The path from `A` to `B` is the tree diameter.
4. Return the middle node or two middle nodes of that path.

Complexity:

```
Time: O(n)
Space: O(n)
```

This is also efficient, but slightly more involved because we need to reconstruct the diameter path.

---

2. Brute Force DFS/BFS from Every Node

For every node, root the tree there and compute its height.

Complexity:

```
Time: O(n^2)
Space: O(n)
```

This is too slow for `n <= 2 * 10^4`.

Key Pattern

This problem is a classic example of:

```
Tree Center Finding
```

The leaf-trimming approach is often called:

```
Topological BFS on an Undirected Tree
```

The final remaining node or two nodes are the centers of the tree and therefore the roots of all Minimum Height Trees.